

Week 04 – Understanding YOLOv26 Meta Files and Dataset Configuration

Introduction

In this task, I explored the directory structure and configuration files used in YOLOv26 datasets. These files are important because they help the model understand where the dataset is stored, what object classes are present, and how annotations are represented during training and validation.

YOLO models depend on properly organized datasets for efficient object detection and segmentation.

YOLO Dataset Structure

A standard YOLO dataset mainly contains:

- images folder
- labels folder
- dataset YAML configuration file

Example structure:

```
dataset/  
  dataset.yaml  
  images/  
    train/  
    val/  
  labels/  
    train/  
    val/
```

Explanation

- images/train → training images
- images/val → validation images
- labels/train → label files for training images
- labels/val → label files for validation images

Each image has a corresponding `.txt` label file with the same filename.

Example:

```
images/train/img1.jpg
labels/train/img1.txt
```

The YOLO loader automatically maps image files to their corresponding label files.

YAML Configuration File

The `dataset.yaml` file acts as the metadata configuration file for the dataset.

Example:

```
path: ../dataset
train: images/train
val: images/val

nc: 3

names:
  0: person
  1: helmet
  2: conveyor
```

Important Parameters

Parameter	Description
path	Root directory of dataset
train	Path to training images
val	Path to validation images
nc	Number of object classes
names	Mapping of class IDs to class names

Observations

- `nc` defines how many classes the model will detect
- `names` provides readable class labels
- train and validation paths help YOLO locate the dataset correctly

YOLO Label File Format

Each image contains a corresponding label file in `.txt` format.

Format:

```
class_id x_center y_center width height
```

Example:

```
0 0.45 0.52 0.30 0.40
```

Meaning of Values

Field	Description
class_id	Object category ID
x_center	X-coordinate of box center
y_center	Y-coordinate of box center
width	Width of bounding box
height	Height of bounding box

All coordinate values are normalized between 0 and 1.

Normalization in YOLO

YOLO uses normalized coordinates instead of pixel values.

Advantages:

- resolution independence
- better scaling during training
- easier handling of different image sizes

Normalization helps the model train effectively even when images are resized.

Relevance to My Final Project

For my proposed project:

Conveyor Nip-Point Hand Intrusion Detection

A custom YOLO dataset can be created using:

- conveyor belt images
- worker hand/glove images
- dangerous nip-point regions

The YAML configuration file would define custom classes, while label files would store bounding box coordinates for training the detection model.

This can help improve industrial worker safety by detecting hand intrusion near hazardous conveyor nip-points in real time.

References

- Ultralytics YOLO Documentation: <https://docs.ultralytics.com/>
- YOLO Dataset Format Guide: <https://docs.ultralytics.com/datasets/detect/>
- COCO Dataset Example: <https://docs.ultralytics.com/datasets/detect/coco8/>